

LABORATORIO SDN

Hombre en el Medio (MITM)

Guía completa de implementación — 4 Fases

GNS3 + VirtualBox + Ryu + Open vSwitch + Scapy

Especialización en Ciberseguridad

Fase	Objetivo	Estado
1 — Entorno	VMs + VirtualBox + OVS + Ryu + conectividad	■ Completada
2 — Ataque	ARP Spoofing + inyeccion de flujos OpenFlow	■ Completada
3 — Evidencias	Capturas PCAP + extraccion flujos + IoC	■ Completada
4 — Mitigacion	Deteccion activa en Ryu + metricas TP/FP	■ En pruebas

1. ARQUITECTURA Y DIRECCIONAMIENTO IP

El laboratorio usa **5 VMs Ubuntu Server 22.04** corriendo en VirtualBox, todas conectadas a la misma **Red Interna de VirtualBox** con nombre **intnet**. No se requiere internet ni DHCP durante el lab.

VM	Interfaz	Direccion IP	Mascara	Red	Rol
ryu	enp0s3	192.168.100.10	/24	Control	Controlador OpenFlow
ovs	enp0s3	192.168.100.20	/24	Control	OVS — canal OpenFlow
ovs	enp0s8	10.0.0.1	/24	Datos	OVS — puertos de hosts
h1	enp0s3	10.0.0.11	/24	Datos	Victima A
h2	enp0s3	10.0.0.12	/24	Datos	Victima B / servidor
h3	enp0s3	10.0.0.13	/24	Datos	Atacante

Plano de control 192.168.100.0/24
ryu (192.168.100.10) ←OpenFlow 1.3 tcp:6653→ ovs (192.168.100.20)

Plano de datos 10.0.0.0/24
h1 (10.0.0.11) ■■
h2 (10.0.0.12) ■■■→ ovs/br0 (10.0.0.1) enp0s8 → br0
h3 (10.0.0.13) ■■

Red VirtualBox: todos los adaptadores en Red interna → **intnet**

Configuracion de adaptadores VirtualBox

■ Configurar con las VMs APAGADAS. El nombre debe ser exactamente **intnet** en todas.

VM	Adaptador VirtualBox	Conectado a	Nombre red	Interfaz Linux
ryu	Adaptador 1	Red interna	intnet	enp0s3
ovs	Adaptador 1	Red interna	intnet	enp0s3
ovs	Adaptador 2	Red interna	intnet	enp0s8
h1	Adaptador 1	Red interna	intnet	enp0s3
h2	Adaptador 1	Red interna	intnet	enp0s3
h3	Adaptador 1	Red interna	intnet	enp0s3

2. PREPARACION DE VMs (una sola vez)

Instala Ubuntu Server 22.04 en una VM base, luego **clona 5 veces** marcando 'Generar nuevas MACs para todos los adaptadores'. Nombres: **ryu**, **ovs**, **h1**, **h2**, **h3**.

Clonar repositorio en cada VM

```
git clone ~/proyecto-mitm
cd ~/proyecto-mitm
chmod +x scripts/*.sh ovs/*.sh hosts/*.sh attack/*.sh mitigation/*.sh
```

VM ryu — Controlador (Python 3.9 + Ryu 4.34)

■ Ryu 4.34 NO funciona con Python 3.10+. Usar Python 3.9 via deadsnakes PPA.

```
sudo apt-get update
sudo apt-get install -y software-properties-common
sudo add-apt-repository -y ppa:deadsnakes/ppa
sudo apt-get update
sudo apt-get install -y python3.9 python3.9-venv python3.9-dev
python3.9 -m venv ~/ryu-venv
source ~/ryu-venv/bin/activate
pip install --upgrade "pip<21" "setuptools==58.2.0" wheel
pip install "eventlet==0.30.2" "ryu==4.34" requests
ryu-manager --version # debe imprimir: ryu-manager 4.34
```

VM ovs — Open vSwitch

```
sudo apt-get update
sudo apt-get install -y openvswitch-switch
sudo systemctl enable openvswitch-switch
ovs-vsctl --version # verifica instalacion
```

VMs h1, h2 — Victimias

```
sudo apt-get update
sudo apt-get install -y tcpdump iproute2 python3-pip
```

VM h3 — Atacante

```
sudo apt-get update
sudo apt-get install -y tcpdump iproute2 python3-pip
# Para instalar netfilterqueue necesitas internet (agregar adaptador NAT temporalmente):
sudo apt-get install -y build-essential python3-dev libnetfilter-queue-dev libnfnetlink-dev
curl netcat
pip3 install netfilterqueue requests scapy
python3 -c "import netfilterqueue; print('OK')"
```

■ Para instalar pip en h3: agrega un Adaptador 2 NAT en VirtualBox temporalmente, instala los paquetes y luego puedes quitarlo. El Adaptador 1 (intnet) no se afecta.

3. FASE 1 — ENTORNO Y CONECTIVIDAD

Estos pasos se repiten **en cada sesion** porque las IPs no son persistentes.

3.1 Asignar IPs (cada vez que se inician las VMs)

VM ryu

```
sudo ip link set enp0s3 up
sudo ip addr add 192.168.100.10/24 dev enp0s3
```

VM ovs

```
sudo ip link set enp0s3 up
sudo ip addr add 192.168.100.20/24 dev enp0s3
sudo ip link set enp0s8 up
sudo ip addr add 10.0.0.1/24 dev enp0s8
```

VM h1

```
sudo ip link set enp0s3 up
sudo ip addr add 10.0.0.11/24 dev enp0s3
```

VM h2

```
sudo ip link set enp0s3 up
sudo ip addr add 10.0.0.12/24 dev enp0s3
```

VM h3

```
sudo ip link set enp0s3 up
sudo ip addr add 10.0.0.13/24 dev enp0s3
sudo sysctl -w net.ipv4.ip_forward=1
```

3.2 Verificar conectividad base

```
# Desde h1 — todos deben responder 0% packet loss:
ping -c3 10.0.0.12 # h1 -> h2
ping -c3 10.0.0.13 # h1 -> h3
ping -c3 10.0.0.1 # h1 -> ovs

# Desde ryu:
ping -c3 192.168.100.20 # ryu -> ovs
```

✓ Si todos responden con 0% packet loss, la conectividad base esta lista.

3.3 Configurar OVS + Ryu (OpenFlow)

En ryu — iniciar controlador

```
source ~/ryu-venv/bin/activate
# Fase 1 basica:
ryu-manager ryu.app.simple_switch_13

# Con API REST (necesario para Fases 2, 3 y 4):
ryu-manager --wsapi-port 8080 ryu.app.simple_switch_13 ryu.app.ofctl_rest
```

En ovs — configurar bridge OpenFlow

```
sudo systemctl start openvswitch-switch
sudo ovs-vsctl add-br br0 # error si ya existe: ignorar
sudo ovs-vsctl add-port br0 enp0s8 # error si ya existe: ignorar
```

```
sudo ovs-vsctl set-controller br0 tcp:192.168.100.10:6653
sudo ovs-vsctl show
# Verificar: is_connected: true
```

■ Si enp0s8 muestra 'master ovs-system': ver seccion de problemas frecuentes al final.

Verificar OpenFlow activo

```
sudo ovs-ofctl -O OpenFlow13 dump-flows br0
# Resultado esperado:
# priority=0 actions=CONTROLLER:65535 (n_packets > 0)
```

✓ Fase 1 completa: OVS conectado a Ryu, OpenFlow 1.3 activo, Ryu muestra 'packet in'.

4. FASE 2 — ATAQUE MITM

La Fase 2 lanza el ataque Man-in-the-Middle desde h3. Ryu debe estar corriendo **con API REST**.

4.1 ARP Spoofing (en h3)

Envenena las cachés ARP de h1 y h2 para que el tráfico entre ellos pase por h3.

```
cd ~/proyecto-mitm
sudo python3 attack/arp_spoof.py --target 10.0.0.11 --gateway 10.0.0.12 --iface enp0s3
# Debe mostrar: [+] Paquetes ARP enviados: N (incrementando)
# Ctrl+C para detener y restaurar las caches
```

4.2 Verificar MITM activo (en h1)

```
ip neigh show
# La MAC de 10.0.0.12 debe ser la MAC de h3, NO la de h2
# Si ambas (10.0.0.12 y 10.0.0.13) tienen la misma MAC -> MITM confirmado
```

4.3 Interceptar trafico (en h3 — segunda terminal)

```
cd ~/proyecto-mitm
sudo python3 attack/mitm_intercept.py --mode sniff --auto-iptables
```

4.4 Generar trafico de prueba

```
# En h2 (servidor):
python3 -m http.server 8000

# En h1 (cliente):
curl http://10.0.0.12:8000
ping -c10 10.0.0.12
```

4.5 Inyeccion de flujos OpenFlow (en ryu o en ovs)

```
cd ~/proyecto-mitm
# Ver flujos actuales:
python3 attack/flow_inject.py --controller 192.168.100.10 list

# Inyectar flujo malicioso (duplicar trafico de h1 al atacante h3):
python3 attack/flow_inject.py --controller 192.168.100.10 inject \
--dpid 1 --src-ip 10.0.0.11 --normal-port 2 --attacker-port 3

# Limpiar flujos maliciosos:
python3 attack/flow_inject.py --controller 192.168.100.10 clear --dpid 1
```

■ Ajusta --normal-port y --attacker-port segun los numeros de puerto que muestre: `sudo ovs-ofctl -O OpenFlow13 show br0`

5. FASE 3 — RECOLECCION DE EVIDENCIAS

Ejecutar con el ARP Spoofing activo. Los artefactos se guardan en **captures/**.

5.1 Captura PCAP + log ARP (en h3)

```
cd ~/proyecto-mitm
# Captura 120 segundos de trafico MITM:
sudo bash scripts/collect_evidence.sh enp0s3 120
# Genera: captures/mitm_capture_.pcap
# captures/arp_events_.log
# captures/evidence_summary_.txt
```

5.2 Estadísticas de flujo OpenFlow (en ovs o ryu)

```
cd ~/proyecto-mitm
# Snapshot unico:
python3 scripts/flow_stats.py --controller 192.168.100.10 --once

# Monitoreo continuo 2 minutos:
python3 scripts/flow_stats.py --controller 192.168.100.10 --interval 5 --duration 120
# Genera: captures/flow_stats_.json
```

5.3 Analisis IoC sobre el PCAP (en cualquier VM con scapy)

```
cd ~/proyecto-mitm
python3 scripts/analyze_ioc.py captures/mitm_capture_.pcap --report
# Genera: captures/ioc_report_.json
```

5.4 IoC que detecta el analizador

IoC	Severidad	Descripcion
ARP_SPOOFING	ALTA	IP con multiples MACs distintas en la captura
MAC_CHANGE	ALTA	Una IP cambia de MAC a lo largo del tiempo
GRATUITOUS_ARP	MEDIA	Respuestas ARP no solicitadas (src_ip == dst_ip)
ARP_FLOODING	MEDIA	Tasa ARP > 3 pkt/s (normal < 1 pkt/s)
MALICIOUS_COOKIE	ALTA	Cookie 0x6d69746d en flujo OpenFlow (inyeccion)
HIGH_PACKET_IN	ALTA	Tasa packet_in > 10 pkt/s al controlador

6. FASE 4 — MITIGACION ACTIVA (en pruebas)

Reemplaza `simple_switch_13` por `arp_monitor.py`, que detecta ARP Spoofing en tiempo real e instala flujos DROP automáticamente.

6.1 Iniciar Ryu en modo mitigacion (en ryu)

```
cd ~/proyecto-mitm
bash mitigation/run_mitigation.sh
# O manualmente:
source ~/ryu-venv/bin/activate
ryu-manager --wsapi-port 8080 mitigation/arp_monitor.py ryu.app.ofctl_rest
```

6.2 Preparar OVS (en ovs) — igual que Fase 1

```
sudo systemctl start openvswitch-switch
sudo ovs-vsctl add-br br0 2>/dev/null || true
sudo ovs-vsctl add-port br0 enp0s8 2>/dev/null || true
sudo ovs-vsctl set-controller br0 tcp:192.168.100.10:6653
```

6.3 Lanzar el ataque (en h3) — mismo que Fase 2

```
sudo python3 attack/arp_spoof.py --target 10.0.0.11 --gateway 10.0.0.12 --iface enp0s3
```

Ryu detectara el cambio de MAC y emitira logs como:

```
WARNING:arp_monitor:ARP_SPOOFING_DETECTED — IP 10.0.0.12 cambio de MAC ...
WARNING:arp_monitor:*** BLOQUEO ACTIVO: MAC 08:00:27:xx:xx:xx bloqueada ***
```

6.4 Verificar bloqueo activo

```
# En ovs — ver flujo DROP instalado:
sudo ovs-ofctl -O OpenFlow13 dump-flows br0
# Buscar: priority=100, actions=drop

# En ryu — ver metricas de mitigacion:
curl -s http://192.168.100.10:8080/mitigation/stats | python3 -m json.tool
curl -s http://192.168.100.10:8080/mitigation/blocked
```

6.5 Evaluacion de metricas

```
cd ~/proyecto-mitm
# Post-ataque:
python3 scripts/evaluate_mitigation.py --log captures/mitigation_events.log --save

# En vivo:
python3 scripts/evaluate_mitigation.py --live --controller 192.168.100.10 --save
# Genera: captures/mitigation_evaluation.json
```

6.6 Detector de anomalias en flujos (en paralelo)

```
python3 mitigation/flow_anomaly.py --controller 192.168.100.10 --interval 5 --duration 120
--report
```

Metrica	Valor objetivo
True Positives (TP)	>= 1 por sesion de ataque
False Positives (FP)	0 (trafico legitimo no bloqueado)

Tiempo de deteccion	< 5 s desde inicio de ARP Spoofing
Tiempo de respuesta	< 1 s desde deteccion hasta flujo DROP
Conectividad h1<->h2	Se restaura despues del bloqueo del atacante

7. PROBLEMAS FRECUENTES Y SOLUCIONES

Error / Sintoma	Causa	Solucion
enp0s8 muestra 'master ovs-system'	OVS control de la interfaz en sesion anterior	ovs-vsctl del-port br0 enp0s8 systemctl stop ovs ip addr flush dev enp0s8 ip addr add 10.0.0.1/24 dev enp0s8 ip link set enp0s8 up
RTNETLINK: File exists al añadir IP	la IP ya estaba asignada	ip addr flush dev enp0s3 luego ip addr add de nuevo
br0 already exists en ovs-vsctl	Bridge de sesion anterior	Ignorar el error, continuar con add-port
version negotiation failed en ovs-vsctl	OpenFlow 1.0 por defecto	siempre -O OpenFlow13 al comando
Network is unreachable al hacer ping	IP asignada en esa VM	Repetir Paso 3.1 en la VM afectada
GNS3 Ethernet Switch no responde	no se pudo reconfigurar adaptador de VM	desactivar adaptador de VM
pip install netfilterqueue falla (DNS error)	no tiene internet	Agregar Adaptador 2 NAT temporalmente, instalar, quitar NAT
is_connected: false en ovs-vsctl show	Ryu no está corriendo o IP de controlador incorrecta	Verificar que ryu corra y que controlador esté activo; revisar IP en set-controller

8. CHECKLISTS DE CIERRE POR FASE

Fase 1

- Todos los pings entre VMs responden con 0% packet loss
- ovs-vsctl show -> is_connected: true
- ovs-ofctl -O OpenFlow13 dump-flows br0 muestra priority=0 actions=CONTROLLER:65535
- Terminal de Ryu muestra 'packet in' de multiples MACs

Fase 2

- arp_spoof.py corre en h3 mostrando '[+] Paquetes ARP enviados: N'
- ip neigh show en h1: MAC de 10.0.0.12 es la MAC de h3
- verify_mitm.sh retorna 'MITM CONFIRMADO'
- mitm_intercept.py muestra trafico ICMP/HTTP entre h1 y h2
- flow_inject.py inject instala flujo con cookie 0x6d69746d en dump-flows

Fase 3

- captures/mitm_capture.pcap existe con trafico capturado
- analyze_ioc.py reporta IoC de severidad ALTA (ARP_SPOOFING o MAC_CHANGE)
- flow_stats.py muestra n_packets > 0 en flujo default
- captures/ioc_report.json generado
- captures/mitigation_events.log o evidence_summary.txt presentes

Fase 4

- arp_monitor.py detecta ARP Spoofing en < 5 s
- OVS instala flujo DROP prioridad 100 para MAC atacante
- evaluate_mitigation.py: TP >= 1, FP = 0
- curl /mitigation/blocked muestra MAC del atacante

